

Assistente de Pesquisa de Engenharia de Dados

Relatório técnico de diagnóstico, solução e validação de coleta de dados do portal G1 para o termo LGPD

Fernanda Fregulha

Vaga
Assistente de Pesquisa de
Engenharia de Dados

Repositório GitHub: <https://github.com/fregulha/netlab-g1-webscraping.git>

Projeto: Diagnóstico e correção de scraper para coleta de resultados de busca do G1

Resumo executivo

Este relatório apresenta a solução desenvolvida para diagnosticar e corrigir a coleta de resultados de busca do portal G1 para o termo LGPD. O projeto foi conduzido com foco em robustez, qualidade dos dados, rastreabilidade e validação automatizada. O objetivo foi identificar a causa da falha da rotina original, corrigir os pontos de fragilidade e entregar um processo de coleta confiável, documentado e verificável para uso em pesquisa e análise de dados.

O trabalho combina análise de HTML, revisão de seletores, normalização de URLs, deduplicação, exportação de arquivos estruturados e execução de testes automatizados. Em termos práticos, a solução reduz riscos de perda de registros, aumenta a confiabilidade dos dados extraídos e demonstra capacidade de agir de forma analítica, técnica e organizada em cenários reais de engenharia de dados e pesquisa.

1. Contexto do problema

A rotina de web scraping original foi usada para coletar resultados da busca do G1 para o termo “LGPD”.

O problema observado foi a regressão do processo: a coleta passou a retornar poucos resultados, registros parcialmente vazios e, em alguns cenários, nenhum item válido. A hipótese levantada pela equipe foi a mudança estrutural da página, os modificadores dos seletores e a mudança de comportamento do carregamento dinâmico.

A análise do código original demonstrou que a rotina priorizava seletores fixos e um modelo de extração estático, sem validação do conteúdo da resposta e sem estratégia de deduplicação. Além disso, a página do G1 passou a exibir resultados em um contexto renderizado dinamicamente, o que impacta o uso de HTML bruto e a lógica de paginação herdada.

2. Diagnóstico técnico

A evidência observada diretamente no navegador confirma que a página de busca do G1 não entrega a totalidade dos resultados em uma única carga HTML estática. Ao rolar a página, o portal dispara requisições de rede e vai montando novos itens dinamicamente na interface. Isso torna o HTML inicial insuficiente para coletar o conjunto completo de resultados, porque parte do conteúdo é carregado após a primeira renderização.

Esse comportamento explica a regressão observada na rotina original: a coleta baseada em HTML bruto da primeira carga passa a retornar poucos resultados, registros vazios ou registros incompletos, sem que a rotina registre falha explícita. Em termos práticos, a solução precisa considerar não apenas seletores

HTML, mas também carregamento incremental via fetch/scroll e a necessidade de sincronizar a coleta com esse ciclo dinâmico.

Adicionalmente, a rotina original apresentava problemas de robustez operacional, como: sobrescrita de listas de resultados, ausência de deduplicação por URL, campos ausentes não tratados, ausência de validação de HTTP e timeout, uso de seletores incompatíveis com a nova estrutura, ausência de logs e falta de quebra clara entre aquisição, parsing e exportação.

3. Estratégias adotadas

A correção foi estruturada em quatro eixos estratégicos: aquisição, parsing, normalização e exportação. O primeiro passo foi fortalecer a etapa de coleta por meio de validação da resposta HTTP, controle de timeout, tratamento de falhas de rede e registros de execução para monitoramento. Essa medida reduz o risco de a rotina falhar silenciosamente em cenários de paginação, instabilidade de rede ou mudanças rápidas na estrutura do portal.

O segundo eixo foi o parsing defensivo e adaptativo. Em vez de depender de seletores frágeis e fixos, o código passou a revisar o HTML real observado nos snapshots, ajustar os caminhos para os elementos presentes no layout atual e extrair apenas os campos que realmente existiam nos cards de busca. A lógica também foi desenhada para tratar valores ausentes e registros incompletos sem quebrar o fluxo de coleta.

O terceiro eixo foi o tratamento do carregamento incremental da página. Como a evidência do navegador mostra que a busca do G1 vai adicionando resultados ao rolar, a solução foi organizada para reconhecer esse padrão e, em versões mais robustas, seguir o comportamento real do portal com automação de navegador ou consumo da API que alimenta os cards. Isso reduz a fragilidade de uma coleta que assume uma página estática e completa desde a primeira resposta.

O quarto eixo foi a normalização e a garantia de integridade dos dados. A URL foi padronizada para remover parâmetros de rastreamento conhecidos, como `utm_*`, `fbclid` e `gclid`, preservando apenas os parâmetros relevantes para a pesquisa. Além disso, foi implementada a deduplicação por URL para evitar duplicidade entre lotes cumulativos, preservando a primeira ocorrência e reduzindo ruído na base final. O pipeline de exportação também foi reforçado com rastreabilidade por página, snapshot, origem e status de processamento.

4. Seletores atualizados e arquitetura da coleta

Os seletores e regras de parsing foram ajustados com base nos snapshots reais levantados. Entre os pontos identificados, destacam-se:

- Card de notícia: `li.widget--info`
- Card de vídeo: `li.video-widget--info`
- Título: `.widget--info__title`
- Resumo: `.widget--info__description`
- Data exibida: `.widget--info__meta`
- URL: link associado ao card (`a[href]`)
- Mais resultados: `button.pagination__load-more`

A rotina também passou a preservar o primeiro lote em que cada registro apareceu, evitando duplicidade em múltiplos snapshots cumulativos.

5. Trechos do código implementado

```
def normalize_url(value, base=BASE):  
    if not value:
```

```

        return None
    p = urlsplit(urljoin(base, value.strip()))
    if p.scheme not in {'http', 'https'}:
        return None
    query = [(k, v) for k, v in parse_qsl(p.query, keep_blank_values=True)
              if not k.lower().startswith('utm_')
              and k.lower() not in {'fbclid', 'gclid'}]
    return urlunsplit((p.scheme.lower(), p.netloc.lower(), p.path,
                      urlencode(query), ''))

def parse_html(html, *, page, collected_at, source_url, snapshot):
    soup = BeautifulSoup(html, 'html.parser')
    cards = soup.select(CARD)
    records = []
    for pos, card in enumerate(cards, 1):
        title = card.select_one('.widget--info__title')
        link = title.find_parent('a') if title else None
        url = normalize_url(link.get('href') if link else None, source_url)
        records.append({
            'titulo': text(title),
            'url': url,
            'resumo': text(card.select_one('.widget--info__description')),
            'data_exibida': text(card.select_one('.widget--info__meta')),
            'pagina': page,
            'coletado_em': collected_at,
            'fonte_url': source_url,
            'snapshot': snapshot,
            'posicao': pos,
        })

```

6. Estratégia de tratamento de erros e robustez operacional

A rotina foi construída para não quebrar completamente diante de problemas comuns em coleta na web.

Os controles implementados incluem:

- timeouts e conexões com retries limitados
- validação de Content-Type antes de processar a resposta
- captura de HTTPError, RequestException e ValueError
- registros em quarentena quando a URL está ausente ou inválida
- saída parcial com código de retorno distinto para falhas controladas
- logs de execução com página processada, quantidade de cards e eventos

7. Validação e evidências de qualidade

Os testes automatizados foram executados com sucesso e a evidência final forneceu 24 testes aprovados em 0,35s. O projeto também inclui base de dados real coletada em lotes cumulativos, extratos de métricas e dados de referência para comparação.

Métrica	Resultado
Testes automatizados	24 aprovados
URLs únicas	30
Notícias	26
Vídeos	4
Deduplicação	30 repetições removidas
Título / URL / resumo / data exibida	Presente em 30/30
data_publicacao	0/30 (campo não inferido com confiança)

8. Estratégia de uso de LLM

A LLM pode ser usada como suporte para manutenção e monitoramento do scraper, mas não deve ser responsável por preencher campos sensíveis ou inventar dados. O uso ideal é como ferramenta de apoio em diagnóstico e revisão.

Em termos de processo, o modelo pode analisar trechos de HTML, comparar versões da página, sugerir seletores alternativos, revisar inconsistências em logs e propor novos testes de regressão. Contudo, qualquer resposta do modelo deve ser validada com amostras reais, snapshots e testes automatizados antes de entrar em produção.

Para evitar alucinações, o modelo deve operar apenas sobre dados observáveis, com a regra de que qualquer sugestão deve estar respaldada por evidência textual do HTML ou pelos registros de execução.

O sistema de produção não deve aceitar dados preenchidos pela LLM sem verificação humana ou automatizada.

9. Limitações e caminhos de evolução

A coleta em ambiente real ainda depende de validação do comportamento de paginação do portal e do uso de um browser ou mecanismo compatível com carregamento dinâmico. A página G1, em muitos momentos, não expõe todos os dados de forma estática; os resultados vão sendo adicionados conforme o usuário rola a página e as requisições de rede são disparadas. Também ficou evidente que a data de publicação não deve ser inferida a partir de URL ou data exibida sem validação em metadados específicos do artigo.

Como melhorias futuras, sugere-se: monitoramento contínuo da estrutura do HTML, inspeção de rede para identificar a API de carregamento, captura de snapshots por período, automatização de checagem de seletores e expansão do módulo de avaliação para métricas de atualização, acurácia por campo e rastreabilidade de origem. Em cenários de cobertura completa, a solução mais robusta é seguir o carregamento real do portal com scroll e automação de navegador.

10. Conclusão

A solução desenvolvida demonstrou que a falha da rotina original estava ligada a combinações de mudança estrutural da página, baixa robustez de parsing e ausência de controles de qualidade. O resultado foi uma coleta mais estável, documentada e rastreável, validada por testes automatizados e por dados reais coletados em lotes cumulativos.

Para uma vaga de Assistente de Pesquisa de Engenharia de Dados, a entrega evidencia capacidade de diagnóstico em ambientes web, uso de Python para extração e processamento de dados, atenção à qualidade e rastreabilidade dos dados e capacidade de propor estratégias para manutenção, avaliação e monitoramento de fluxos de coleta.

Links e referências

GitHub: <https://github.com/fregulha/netlab-g1-webscraping.git>

Portal alvo: <https://g1.globo.com/busca/?q=lgpd>

Validação automatizada: 24 testes aprovados com pytest